

Backend Depth Roadmap

Multi-Tenant Event Ticketing SaaS — from CRUD-competent to senior-level backend engineering

The project: a multi-tenant event ticketing backend. Every advanced concern is a real requirement, not a bolted-on exercise — seat booking forces concurrency control, payments force idempotency, dashboards force query optimization, multiple organizations force auth and data isolation, and confirmations/QR generation force async queueing. Build the phases in order: shortcuts in an early phase become bugs in a later one.

Guardrail: stay in Node.js + PostgreSQL/MongoDB. Don't reach for a new framework, a new language, or microservices to feel like you're 'going deep' — that resets the learning curve instead of extending it. A single well-structured monolith that handles all six phases correctly is a stronger signal than a distributed system with fuzzy boundaries.

Phase 1: Data Model & Multi-Tenancy Foundation

Get the one decision right that everything else depends on: how tenants are isolated.

- Model orgs, events, seats, users, bookings.
- Decide tenancy strategy up front: shared tables + `tenant_id` column vs. schema-per-tenant. Write down the reasoning — this is expensive to reverse later.
- Implement Postgres Row-Level Security (RLS) so isolation is enforced at the database layer, not only in application query code.
- Try to break your own isolation with a raw query that 'forgets' the tenant filter, and confirm RLS still blocks it.

Downside of skipping RLS and relying only on `WHERE tenant_id = ?` in app code: one forgotten clause in one endpoint leaks another org's data — and you won't find out until someone else does.

Phase 2: Seat Booking Under Concurrency

This is where 'CRUD backend' stops being enough — seats are a finite, contested resource.

- Implement seat booking against limited inventory per event.
- Choose a locking strategy based on expected contention — don't default to whichever you read about first: `SELECT FOR UPDATE` (pessimistic), optimistic locking with a version column, or Postgres advisory locks.
- Load-test it yourself: fire concurrent booking requests against a small seat pool and confirm you never oversell.

Downside of not load-testing this yourself: overselling under concurrency is invisible in manual testing and only shows up under real traffic — exactly when it's most expensive to discover.

Phase 3: Payments With Idempotency

Build this assuming the network is hostile.

- Add payment confirmation before a booking is finalized.
- Handle client retries on timeout without double-charging, using an idempotency key with a unique constraint at the database level.
- Think through the hard case: the external gateway times out on the client's end but actually succeeded — what happens when the client retries?
- Handle two 'identical' requests racing at the database level at the same millisecond.

Downside of relying on an idempotency key alone without a DB-level uniqueness guarantee: it only protects against retries you handle in application code, not races — a naive implementation looks correct until two requests land in the same instant.

Phase 4: Auth With Real Revocation

Different permission levels, and revocation that actually works — the part most implementations fake.

- Support org owners, staff, and customers as distinct permission levels.
- Use short-lived access tokens plus revocable refresh tokens.
- Implement refresh token rotation with reuse detection: if an already-rotated refresh token is replayed, kill the entire session chain — that's the signal of a stolen token.

Downside of plain JWTs with no rotation/revocation: a stolen refresh token works forever, silently, and you have no way to detect or stop it without a DB lookup on every single request — which defeats the point of using JWTs in the first place.

Phase 5: Async Work: Queues & Backpressure

Confirmation emails, QR generation, and analytics shouldn't block the request/response cycle.

- Add a job queue (e.g. BullMQ + Redis) for emails, ticket QR generation, and post-event analytics.
- Deliberately push more load through it than your workers can handle and watch how it degrades.
- Implement retries with exponential backoff, a dead-letter queue for permanently failed jobs, and basic visibility into queue depth/health.

Downside of skipping the overload test: a queue that works fine at low volume can silently back up or OOM under load, and without dead-letter handling, failed jobs vanish instead of surfacing as a problem you can fix.

Phase 6: Read-Path Optimization

Build an organizer dashboard once there's real data volume — this is where N+1 and missing indexes show up naturally.

- Build a dashboard: bookings per event, revenue, occupancy.
- Profile before optimizing — measure which query is actually slow instead of guessing.
- Fix N+1 query patterns with proper joins/aggregation; use EXPLAIN ANALYZE to confirm indexes are actually being used by the planner.

Downside of optimizing without profiling first: you'll likely tune the wrong query, ship added complexity for no measured benefit, and miss the actual bottleneck.

Working Rules

- Do the phases in order — don't skip to the interesting ones. Each phase's shortcuts become the next phase's bugs.
- No microservices, no framework-hopping. Depth in one monolith beats breadth across a distributed system.
- Write a short postmortem after each phase: what broke first, why, and what you'd do differently. That's what actually builds seniority — the code is just the vehicle.
- Come back to this conversation when a phase is done for the next step.